on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Summary

The LLM prompts were developed in the Rails codebase to leverage existing Ruby expertise and as the AI Gateway was evolving. Now that the AI Gateway is a stable and key part of the GitLab infrastructure the prompts can be migrated from Rails into the AIGW. The Rails monolith remains the persistence and control layer, with AI features becoming a thin entrypoint which refer to the prompt and wrapper code in the AI Gateway. The Rails monolith also has to pass all parameters required for the referenced prompt. Prompt definitions are moved to YAML files on the AI Gateway with Python wrappers. GitLab AI functionality is expected to be unchanged during evaluations.

Motivation

Moving prompts to the AI Gateway offers the following advantages:

- Native access to data science libraries written in Python
- Ability to iterate on AI features and prompts without changing Ruby code and upgrading GitLab Rails
- The clients with direct access to the AI Gateway don't need to rely on Rails to retrieve prompts or duplicate the prompt logic
- Ability to maintain or analyze the prompts data that is now stored in a single place

Goals

- Migrate most of the prompts from GitLab Rails Ruby code to YAML files in the AI Gateway
- Preserve the existing AI functionality: product coverage, performance, observability
- Cleanup the prompts that are no longer used (example)

Proposal

Use Agents to implement the functionality that executes a model request based on the given information and agent definition. The agent definition is stored in a YAML file: prompt template, model and client information, and LLM params.

Agent
is an AI-driven entity that performs various tasks. In the context of this blueprint, we refer to the agents implemented in the AI Gateway: an entity that basically locates the defined prompt template and executes it with the passed parameters.

The agents functionality can be exposed by using a generic endpoint, defining a separate endpoint or extending the existing endpoints to use agents.

Use generic endpoint

Generate issue description uses the generic v1/agents/{agentid} endpoint that accepts agent id and the parameters.

Agent id indicates the location of the prompt definition. For example, v1/agents/chat/explaincode expects to find a prompt in the aigateway/agents/definitions/chat/explaincode folder. If a new version of a prompt is introduced, it can be accessed through a new endpoint. For example, v1/agents/chat/explaincode/v1 looks for a definition in the aigateway/agents/definitions/chat/explaincode folder.

The parameters are sent directly to the prompt template of the agent definition. If any parameter is missing, an error is raised, therefore, if the prompt template is changed to include a new parameter, it's a breaking change and a new version of a prompt is recommended.

Define separate endpoint

Duo Chat React uses v2/chat/agent because it's a complex feature that requires pre and post processing.

The prompt version can be controlled by parameters passed to the endpoint.

Extend existing endpoint

Code Completions extends the existing v2/code/completions endpoint to use agents. It enables gradual migration of complex features with a lower risk of breaking the existing functionality.

The prompt version can be controlled by existing or new parameters passed to the endpoint.

Iteration plan

Prompt Migration to AI Gateway is used to track the progress.

- Migrate Code Completions/Generations prompts for Custom Models. The features backed by Custom Models are experimental/beta, i.e lower risk of degrading the experience of existing customers.
  - Migrate Code Completions/Generations prompts for GA Models.
- Migrate Duo Chat ReAct prompt (currently in progress).
  - Migrate Duo Chat ReAct prompt for Custom Models
- Migrate Duo Chat Tools prompts for Custom Models. The features backed by Custom Models are experimental/beta, i.e lower risk of degrading the experience of existing customers.
  - Migrate Code Completions/Generations prompts for GA Models.
- Migrate other Duo Features.

Design and implementation details

Prompt Definition

Agents are defined in AI Gateway at aigateway/agents/definitions. The definitions are .yml files stored in a folder per feature: generateissuedescription or chat/react or codesuggestions/generations/v2. The folder is the agent-id.

The name of YAML files is either the name of the model for which the prompt is defined or base.yml: codesuggestions/completions/codegemma.yml or chat/react/base.yml. If an agent

definition for a specific model is requested by passing model metadata, then a definition for the model is used; otherwise, the base definition is used.

This folder structure also supports versioning. For example, v2 subfolder can be created in a feature folder and contain new prompts for all models: codesuggestions/generations/v2/base.yml and codesuggestions/generations/v2/mistral.yml.
The feature then uses codesuggestions/generations/v2 instead of codesuggestions/generations as agent-id to point to the new prompt based on a condition, for example, a feature flag.

As a result, the definitions are stored in the following structure:

yaml
aigateway/agents/definitions

```
chat
  react
    base.yml
    mistral.yml
  explaincode
    base.yml
    mistral.yml
codesuggestions
  completions
    base.yml
    codegemma.yml
    codestral.yml
  generations
    v2
      base.yml
      mistral.yml
    base.yml
    mistral.yml
...
```

This structure has the following benefits:

- Related features can be grouped. For example (code-completions and code-generations)
- A feature can contain multiple versions of a prompt in a folder
- Ambiguity can be resolved by putting features with identical names in different folders: for example, explain-code tool and explain-code feature

The definitions can be potentially improved by introducing inheritance. When a feature has mostly the same definition for all models, it can inherit from or include a base definition and extend it.

Versioning

By versioning our prompts, we allow feature developers to pin the prompt they are using to an

immutable value, enabling other developers to safely work on iterations with the guarantee that this will not cause regressions. For example, Custom Models prompts for Duo Chat on Mistral prompts were broken due to changes to upstream Claude prompts.

Why Semantic Versioning

We use semantic versioning for versioning our prompts, where each version is a file within the target prompt. Using semantic version enables us to communicate expectations about compatibility:

- A bump to the patch means a fix to the prompt that is backwards compatible.
  - Example: removing a rogue \n
- A bump to the minor means a feature addition that doesn't require any changes to the api
  - Example: a new parameter is added to the template but a default is provided
- A bump to the major means a non-backwards compatible change:
  - Example: the template prompt new parameters but providing a default is not possible.

Since this versions are pinned on consumers of the prompts, new iterations will not affect existing released features.

Another benefit we get by using semantic versioning is the extensive tooling for resolving a version. Intead of receiving a specific version, AIGW can resolve a version based on a spec (for example, 1.x would fetch the highest stable version with major being 1).

Version structure

If we have a version 1.0.0 and 2.0.0 for prompts support completion for gpt, versions 1.0.0 and 1.0.1 for claude3 and only 1.0.0 for the default model, then the versioned prompt directory will look like:

```
yaml
definitions/
  codesuggestions/
    completion/
      gpt/
        1.0.0.yml
        1.0.0-rc.yml
        2.0.0.yml
      claude3/
        1.0.0.yml
        1.0.1.yml
    partials/
      completion/
        user/
          1.0.0.yml
```

Pinning a version

Clients should provide a specific range of the prompt to indicate which updates they want to

use automatically (patch, minor). For most AI features, this means setting a version range in the Rails app (e.g. ^1.0), which also allows us to control version changes using feature flags. Some other features, like code completions, will require setting the range in their respective clients (e.g. the VSCode extension, the GitLab Language Server, etc).

Releasing new versions and version expectations

Release candidates (-alpha, -beta, -rc) are ignored by version resolvers, and must be mentioned manually. They are not required to be immutable, which makes them useful for testing a new feature or fix behind a feature flag:

```
ruby
def promptversion
  return '1.0.1-rc' if Feature.enabled?(:featurefix)

  '^1.0'
end
```

This ensures that self-hosted instances can still receive tested updates: self-hosted will only fetch stable versions, as well as evaluations with CEF.

Once results with the release candidate prompt match the expectations, the suffix can be removed. At this point, the version becomes immutable. This can be done once usage was tested (ideally with a feature flag) AND evaluations were run and taken into account.

Some prompts also use template partials to reuse parts across different features: these must also be versioned, since changing them can affect multiple different features. To release a new version of a prompt partial, first create a release version of the partial, and mention that partial in the a new release version for the main prompt.

Migration process

This change requires no action by feature teams initially. The change will be automated: the current prompt directory will be migrated automatically, and every prompt will be assigned 1.0.0 as initial version. The version requested by clients, when not provided, will be 1.0.0 as well. That way, changes initially transparent to feature teams. Once the migration to prompt versioning takes place however, the versioning expectations will be enforced.

Migration work is highlighted in this epic

Disadvantages

- We do not have diffing between consecutive versions in GitLab. We can still diff between files using command line.

- The immutability of a prompt file might not fit our workflow, and require too many new files to be created. As alternative, we can relax the requirement for patches, and just create new files at the minor updates.

Both downsides can be tackled by moving prompts to it's own repository, so that version updates become new commits instead of new files.

Code Completion

Current behavior

Code Completions request either:

- Goes through Rails to generate a prompt and sends it to the AI Gateway
- Goes to the AI Gateway directly if direct access is enabled

```
mermaid
sequenceDiagram
    participant Client
    participant Rails
    participant AIGateway
    participant Model

    Client ->> AIGateway: POST /v2/code/completions with a prompt
    Client ->> Rails: POST /api/v4/codesuggestions/completions
    Rails ->> AIGateway: POST /v2/code/completions with a prompt
    AIGateway ->> Model: sends a prompt
```

Proposal

Code Completions sends an empty or nil prompt and additional data to indicate that the prompt must be generated by the AI Gateway. The AI Gateway uses the request data to generate a prompt itself and sends it to a model:

```
mermaid
sequenceDiagram
    participant Client
    participant Rails
    participant AIGateway
    participant Model

    Client ->> AIGateway: POST /v2/code/completions with empty prompt
    Client ->> Rails: POST /api/v4/codesuggestions/completions
    Rails ->> AIGateway: POST /v2/code/completions with empty prompt
    AIGateway ->> Model: generates a prompt and sends it
```

PoC

- This MR demonstrates extending the existing /v2/code/completions that uses agents to build and execute the prompt.
- This collaboration issue contains more details for using the endpoint.

Code Generation

Current behavior

Code Generations requests go through Rails to generate a prompt and send it to the AI Gateway:

```
mermaid
sequenceDiagram
    participant Client
    participant Rails
    participant AIGateway
    participant Model

    Client ->> Rails: POST /api/v4/codesuggestions/generations with an instruction
    Rails ->> AIGateway: POST /v2/code/generations with a prompt built from the instruction
    AIGateway ->> Model: sends a prompt
```

Proposal

Code Generation sends a request that contains user instructions only and additional data and the AI Gateway generates a prompt to send it to a model.

```
mermaid
sequenceDiagram
    participant Client
    participant Rails
    participant AIGateway
    participant Model

    Client ->> Rails: POST /api/v4/codesuggestions/generations with an instruction
    Rails ->> AIGateway: POST /v2/code/generations[agnetid: <agentid>] with necessary data
    AIGateway ->> Model: generates a prompt and sends it
```

For Code Generations, we can use the prompt field to pass the additional information for code generation, so we cannot nullify it to indicate agent usage:

- Use agentid fields to indicate agent usage with the location of the prompt
- Eventually, prompt field contains the user instruction only
- For the first iterations, we can pass the whole prompt and then iteratively migrate different parts from the Rails prompt to the AI Gateway

PoC

- This PoC demonstrates extending the existing /v2/code/generations that uses agents to build and execute the prompt.
- This collaboration issue contains more details for using the endpoint.

Duo Chat Tools

Current behavior

Rails receives from AI Gateway the information about which tool to invoke, generates a prompt and sends it to AI Gateway.

```
mermaid
sequenceDiagram
    participant Rails
    participant AI Gateway
    participant LLM

    Rails ->> AI Gateway: POST /v2/chat/agent
    AI Gateway ->> LLM: Creates and sends ReAct Prompt
    LLM -->> AI Gateway: Responds with the right tool to invoke
    AI Gateway -->> Rails: Responds with tool to invoke

    Rails ->> AI Gateway: POST /v1/chat/agent with a prompt
    AI Gateway ->> LLM: Propagates the prompt
    LLM -->> AI Gateway: Response
    AI Gateway -->> Rails: Response
```

Proposal

Rails receives from AI Gateway the information about which tool to invoke, sends all related data to generate a prompt to AI Gateway. AI Gateway generates a prompt and sends a request to LLM.

```
mermaid
sequenceDiagram
    participant Rails
    participant AI Gateway
    participant LLM

    Rails ->> AI Gateway: POST /v2/chat/agent
    AI Gateway ->> LLM: Creates and sends ReAct Prompt
    LLM -->> AI Gateway: Responds with the right tool to invoke
    AI Gateway -->> Rails: Responds with tool to invoke

    Rails ->> AI Gateway: POST /v1/agents/tools/<tool-name> with related data
    AI Gateway ->> LLM: Create a prompt and send it
    LLM -->> AI Gateway: Response
    AI Gateway -->> Rails: Response
```

When a new version of a prompt is introduced (like aigateway/agents/definitions/chat/explaincode/v1), then /v1/agents/tools/<tool-name>/<version> endpoint will be called.

PoC

These Rails
and AI Gateway MRs
demonstrate the execution of a chat tool via agents.

Migrating any other tools comes down to:

- Defining unit primitive and creating a feature flag in
  Rails
- Adding a prompt in AI Gateway
- Cleaning up the Rails part after the feature flag is enabled

Testing and Validation Strategy

Ideally, the migration shouldn't change the prompt or any LLM parameters. That's why testing and validation strategy comes down to verifying that the requests to the model are identical before and after the migration.

For Anthropic models, run the AI Gateway with the following env variable and verify that the parameters sent to the Anthropic server are the same:

bash
ANTHROPICLOG=debug poetry run aigateway

For LiteLLM models, run the
proxy
with detailed debug enabled and
verify that the parameters sent to the model are the same:

bash
litellm --detaileddebug

If the prompt or the LLM parameters are changed, then an additional evaluation is recommended before rolling out
(example).

Rollout Plan

The rollout plan depends on the individual feature, but the following collaboration issues can be used as examples:

- Code Generations
- Code Completions

The changes should be introduced behind a feature flag:

- If the features are experimental/beta and can be grouped into a single logical section (like Custom Models), a single feature flag can be used.
- If a feature is GA, a separate feature flag per feature is recommended.

SECTION: engineering/architecture/design-documents/rails_upgrade/_index.md

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Summary

Rails version upgrade is driven by volunteers. The work is usually prioritized when a particular event related to it happens:

- End of life dates
- Next Rails version provided next Ruby version support (Ruby 3 Upgrade)
- Next Rails version has features that a particular group is interested in and it's more feasible to upgrade Rails rather than patch it

The process usually consists of the following steps:

- Rails is bumped to the next version in a draft merge request
- The failed CI jobs and tests are being fixed by pushing commits to the merge request
- The commits are extracted from the merge request into separate MRs to merge them into the default branch
  - Sometimes it's not possible because the changes are applicable only to the next Rails version only (like changed/removed patches)
- When the diff is as small as possible, we consider merging:
  - With Rails 7 upgrade, Delivery suggested and executed experimental rollouts: the commit that contained Rails 7 changes have been cherry-picked into auto-deploy and deployed on gprd-cny and then gprd for a couple of hours

Motivation

With the current process, the following challenges are faced:

- Merging the main MR must be coordinated with Delivery to execute experimental rollouts and reduce the scope of risky changes. Examples of blockers:
  - Delivery group capacity
  - Ruby version upgrade
  - GitLab·s major release
- The green CI of the MR must be reached and maintained while the target branch quickly moves forward. It makes achieving the green CI a moving target.
  - Rails 7.1 MR was green 6 months ago, but its release was postponed due to the blockers above. When the MR was rebased 1 month ago, the CI failed with ~80 failed jobs and ~200 tests, which required significant effort to make it green again.
- Even though, the diff of the MR is as small as possible, it's still sizeable and contains changes in different areas:
  - Rails 7.1 MR changes ~100 files
- It is hard to parallelize and synchronize the engineering effort
  - Engineers have to collaborate in a single MR or have separate ones and compare changes

The main goal is to facilitate the process of addressing the challenges above.

Proposal

The proposal is to include the current and the next Rails versions in Gemfile and maintain a single codebase for both versions. The gem version and differences in the app behavior are controlled by an environment variable. The approach has the following benefits:

Development

- The process doesn't require to have a single large MR anymore. The next Rails version is introduced and the fixes are merged in the main codebase one by one.
- The fixes are introduced and merged via small merge requests even if incompatible with the current Rails version.
- The issues are fixed in parallel by different teams in their area of expertise and the changes are synchronized in the main branch as per usual development

Quality

- CI pipeline that runs tests and checks against the next Rails version can be introduced
  - The pipeline controls that no new failures appear, so achieving the green CI for a particular Rails version is not a moving target anymore
  - The pipeline synchronizes all the fixes introduced by multiple teams and reflects the current state of the app running the next Rails version

Delivery

- Merging the changes related to the next Rails version is no longer coordinated with Delivery. The changes are merged but executed only when the environment variable is switched to enable the next Rails version.
- The experimental rollout process is simplified to switching an environment variable. It enables an easier rollback to the previous Rails version if an issue is encountered.

Design and implementation details

To implement the proposal, we can specify two rails versions in Gemfile:

```
ruby
def next?
  File.basename(FILE) == "Gemfile.next"
end

if next?
  gem 'rails', '~> 7.1.3.4', featurecategory: :shared
else
  gem 'rails', '~> 7.0.8.4', featurecategory: :shared
end
```

Gemfile.next is a symlink to Gemfile. After running

```
bash
BUNDLEGEMFILE=Gemfile.next BUNDLECACHEPATH=vendor/cache.next bundle install
BUNDLEGEMFILE=Gemfile.next BUNDLECACHEPATH=vendor/cache.next bundle exec bundler-checksur
```

we have the following set of files:

```
bash
Gemfile
Gemfile.lock
Gemfile.checksum
Gemfile.next
Gemfile.next.lock
Gemfile.next.checksum
```

The different paths in the codebase are controlled by Gitlab.nextrails? flag:

```
ruby
def self.nextrails?
  return @nextbundlegemfile unless @nextbundlegemfile.nil?

  @nextbundlegemfile = File.exist?(ENV["BUNDLEGEMFILE"]) && File.basename(ENV["BUNDLEGEMFILE
== "Gemfile.next"
end
```

When a test is fixed for both the current and the next Rails versions, it can be verified as:

```
bash
bundle exec rspec <testfilespec.rb>
BUNDLEGEMFILE=Gemfile.next BUNDLECACHEPATH=vendor/cache.next bundle exec rspec
<testfilespec.rb>
```

PoC

Here are the two MRs with identical codebase:

- The one that runs tests with 7.0.8.4 version enabled (by default).
- The one that runs tests with 7.1.3.4 version enabled (by specifying BUNDLEGEMFILE environment variable).

Both pipelines are green.

Analysis

- ~40 Gitlab.nextrails? entries for 7.0.8.4 -> 7.1.3.4 update
  - It·s not a big number for a large codebase like GitLab Rails
  - All the checks are removed after the next Rails version is deployed, i.e the number of

Gitlab.nextrails? entries is reset to zero after every successful upgrade. It allows us to have a retrospective after an upgrade and easily change the approach if we don·t want to use it anymore.
- bundler-checksum is modified to create Gemfile.next.checksum when it's executed for Gemfile.next
- The local gems (in the gems directory) compatibility should be investigated because some of them patch rails gems (like activerecord-gitlab). The pipeline passes in the CI but it should be investigated whether the result is reliable.
- When a gem is updated, both Gemfile.lock and Gemfile.next.lock must be updated. It must be done manually at the moment, we can consider facilitating this process.

Iteration plan

Rails 7.1 Upgrade

The main goal is to keep the pipeline of 7.0.8.4 -> 7.1.3.4 MR green.

- Extract the changes from the main MR or id-next-rails PoC and merge them until all the changes from the main MR are in the main branch.
- Define a CI pipeline that runs the test suite against the next Rails version. The pipeline is not allowed to fail to maintain the green status of 7.1.3.4 version.
- Coordinate with Delivery the date of the rollout.
- Remove the conditional Gitlab.nextrails? checks after the successful rollout
- Evaluate whether the approach is feasible and can be used for future upgrades
- Update the documentation

Future Rails upgrades

The implementation details are to be discussed, but the potential process can be:

- Fix the deprecation warnings
- Introduce the next Rails version to the codebase and fix all the issues on booting
- Define a CI pipeline that runs the test suite against the next Rails version. The pipeline is allowed to fail only for existing failures. The new failures fail the pipeline.
- Fix the issues and merge them to the main branch until the next Rails version pipeline is green.
- Coordinate with Delivery the date of the rollout.

SECTION: engineering/architecture/design-documents/rapid_diffs/_index.md

{{< engineering/design-document-header >}}

Summary

Diffs at GitLab are spread across several places with each area using their own method. We are aiming
to develop a single, performant way for diffs to be rendered across the application. Our aim here is

to improve all areas of diff rendering, from the backend creation of diffs to the frontend rendering
the diffs.

All the diffs features related to this document are listed on a dedicated page.

Work breakdown

Rapid Diffs work is split into 3 stages and can be tracked in the following epics:

1. Stage 0 · foundation:
   - Have foundational components in place.
   - Stream diffs on MR, commit and compare revisions pages.
1. Stage 1 · baseline features:
   - Most of the features are working (dicussions, navigation, review, etc.)
1. Stage 2 · production ready:
   - Feature specs pass against Rapid Diffs
   - Full accessibility compliance

Motivation

Goals

- improved perceived performance
- improved maintainability
- consistent coverage of all scenarios

Non-Goals

<!--
Listing non-goals helps to focus discussion and make progress. This section is optional.

- What is out of scope for this blueprint?
-->

This effort will not:

- Identify improvements for the current implementation of diffs both in Merge Requests or in the Repository Commits

Priority of Goals

In an effort to provide guidance on which goals are more important than others to assist in making
consistent choices, despite all goals being important, we defined the following order.

Perceived performance is above improved maintainability is above consistent coverage.

Examples:

- a proposal improves maintainability at the cost of perceived performance: · we should consider an alternative.
- a proposal removes a feature from certain contexts, hurting coverage, and has no impact on perceived performance or maintainability: · we should re-consider.
- a proposal improves perceived performance but removes features from certain contexts of usage: · it's valid and should be discussed with Product/UX.
- a proposal guarantees consistent coverage and has no impact on perceived performance or maintainability: · it's valid.

In essence, we'll strive to meet every goal at each decision but prioritise the higher ones.

Process

Workspace & Artifacts

- We will store implementation details like metrics, budgets, and development & architectural patterns here in the docs
- We will store large bodies of research, the results of audits, etc. in the wiki of the RRD project
- We will store audio & video recordings on the public YouTube channel in the Code Review / RRD playlist
- We will store drafts, meeting notes, and other temporary documents in public Google docs

Proposal

<!--
This is where we get down to the specifics of what the proposal actually is,
but keep it simple! This should have enough detail that reviewers can
understand exactly what you're proposing, but should not include things like
API designs or implementation. The "Design Details" section below is for the
real nitty-gritty.

You might want to consider including the pros and cons of the proposed solution so that they can be
compared with the pros and cons of alternatives.
-->

The new approach proposed here changes what we have done in the past by doing the following:

1. Stop using virtualized scrolling for rendering diffs.
1. Move most of the rendering work to the server.
1. Enhance server-rendered HTML on the client.
1. Unify diffs codebase across all pages rendering diffs (merge request, repository commits, compare revisions and any other).

Definitions

Maintainability

Maintainable projects are simple projects.

Simplicity is the opposite of complexity. This uses a definition of simple and complex described by Rich Hickey in "Simple Made Easy" (Strange Loop, 2011).

- Maintainable code is simple (single task, single concept, separate from other things).
- Maintainable projects expand on simple code by having simple structure (folders define classes of behaviors, e.g. you can be assured that a component directory will never initiate a network call, because that would be conflating visual display with data access)
- Maintainable applications flow out of simple organization and simple code. The old saying is a cluttered desk is representative of a cluttered mind. Rigorous discipline on simplicity will be represented in our output (the product). By being strict about working simply, we will naturally produce applications where our users can more easily reason about their behavior.

Done

GitLab has an existing definition of done which is geared primarily toward identifying when an MR is ready to be merged.

In addition to the items in the GitLab definition of done, work on RRD should also adhere to the following requirements:

- Meets or exceeds all metrics
  - Meets or exceeds our minimum accessibility metrics (these are explicitly not part of our defined priorities, because they are non-negotiable)
- All work is fully documented for engineers (user documentation is a requirement of the standard definition of done)

Acceptance Criteria

To measure our success, we need to set meaningful metrics. These metrics should meaningfully and positively impact the end user.

1. Meets or exceeds WCAG 2.2 AA.
1. Meets or exceeds ATAG 2.0 AA.
1. The RRD app loads less than or equal to 300 KiB of JavaScript (compressed / "across-the-wire")¹.
1. The RRD app loads less than or equal to 150 KiB of markup, images, styles, fonts, etc. (compressed / "across-the-wire")¹.
1. The Time to First Diff (mr-diffs-mark-first-diff-file-shown) happens before 3 seconds mark.
1. The RRD app can execute in total isolation from the rest of the GitLab product:
   1. "Execute" means the app can load, display data, and allows user interaction ("read-only").
   1. If a part of the application is only used in merge requests or diffs, it is considered part of the Diffs application.
   1. If a part of the application must be brought in from the rest of the product, it is not considered part of the Diffs load (as defined in metrics 3 and 4).
   1. If a part of the application must be brought in from the rest of the product, it may not block functionality of the Diffs application.
   1. If a part of the application must be brought in from the rest of the product, it must be loaded asynchronously.
   1. If a part of the application meets 5.1-5.5 (such as: the Markdown editor is loaded

asynchronously when the user would like to leave a comment on a diff) and its inclusion causes a budget overflow:

- It must be added to a list of documented exceptions that we accept are out of bounds and out of our control.
- The exceptions list should be addressed on a regular basis to determine the ongoing value of overflowing our budget.

---

¹: The Performance Inequality Gap, 2023

Frontend

Ideally, we would meet our definition of done and our accountability metrics on our first try. We also need to continue to stay within those boundaries as we move forward. To ensure this, we need to design an application architecture that:

1. Is:
   1. Scalable.
   1. Malleable.
   1. Flexible.
1. Considers itself a mission-critical part of the overall GitLab product.
1. Treats itself as a complex, unique application with concerns that cannot be addressed as side effects of other parts of the product.
1. Can handle data access/format changes without making UI changes.
1. Can handle UI changes without making data access/format changes.
1. Provides a hookable, inspectable API and avoids code coupling.
1. Separates:
   - State and application data.
   - Application behavior and UI.
   - Data access and network access.

Design and implementation details

Overview

Reusable Rapid Diffs introduce a change in responsibilities for both frontend and backend.

The backend will:

1. Prepare diffs data.
1. Highlight diff lines.
1. Render diffs as HTML and stream them to the browser.
1. Embed diffs metadata into the final response.

The frontend will:

1. Enhance existing and future diffs HTML.
1. Handle streamed diffs HTML.
1. Enhance diffs HTML with dynamic controls to enable user interaction.

Static and dynamic separation

To achieve the separation of concerns, we should distinguish between static and dynamic UI on the page:

- Everything that is static should always be rendered on the server.
- Everything dynamic should be enhanced on the client.

Data that should be coming with the page:

- Static diff file metadata: viewer type, added and removed lines, etc.
- Edit permissions

Data that should be served through additional requests:

- Discussions
- File browser tree
- Line expansion HTML
- Full file HTML
- Code quality
- Code coverage
- Everything else

We should return HTML for line expansion and view full file features.
Other requests should return normalized data in JSON format.

Code suggestion feature should use the existing HTML of the diff, similar to the current implementation.

Performance optimizations

To improve the perceived performance of the page we should implement the following techniques:

1. Limit the number of diffs rendered on the page at first.
1. Use HTML streaming
   to render the rest of the diffs.
   1. Use Web Components to hook into diff files appearing on the page.
1. Apply content-visibility whenever possible to reduce redraw overhead.
1. Render diff discussions asynchronously.

Page & Data Flows

These diagrams document the flows necessary to display diffs and to allow user interactions and user-submitted data to be gathered and stored.
In other words: this page documents the bi-directional data flow for a complete, interactive application that allows diffs to display and users to collaborate on diffs.

Critical Phases

1. Gitaly

1. Database
1. Diff Storage
1. Cache
1. Back end
1. Web API
1. Front end

mermaid

```
flowchart LR
    Gitaly
    DB[Database]
    Cache
    DS[Diff Storage]
    FE[Front End]
    Display

    Gitaly <--> BE
    DB <--> BE
    Cache <--> BE
    DS <--> BE
    BE <--> API
    API <--> FE
    FE --> Display

    subgraph Rails
    direction LR
        BE[Back End]
        API[Web API]
    end
```

[\]: Front end obscures many unexplored phases. It is likely that the front end will need caches, databases, API abstractions (over sub-modules like network connectivity, etc.), and more. While these have not been expanded on, "Front end" stands in for all of that complexity here.

Gitaly

For fetching Diffs, Gitaly provides two basic utilities:

1. Retrieve a list of modified files with associated pre- and post-image blob IDs for a set of revisions.
1. Retrieve a set of Git diffs for an arbitrary set of specified files using pre- and post-image blob IDs.

mermaid

```
sequenceDiagram
    Back end ->> Gitaly: "What files were modified between<br />this pair of/in this single
```

revision?"
    Gitaly ->> Back end: List of paths
    Back end ->> Gitaly: "What are the diffs for this set of paths
 between this pair of/in this single revision?"
    Gitaly ->> Back end: List of diffs

Database

mermaid
sequenceDiagram
    Back end ->> Database: What are the file paths for a known MR version?
    Database ->> Back end: List of paths

Cache

- Fresh render of a diff

mermaid
sequenceDiagram
    Back end ->> Cache: Give me the diff template for scenario XYZ
    Cache ->> Back end: Static template to render diff in scenario XYZ

- Repeated render of a diff

mermaid
sequenceDiagram
    Back end ->> Cache: Give me the compiled UI for diff ABC123
    alt Cache miss
        Cache ->> Back end: ··
        Back end ->> Cache: Cache the compiled UI for diff ABC123
    else
        Cache ->> Back end: Existing compiled diff UI
    end

Diff Storage

mermaid
sequenceDiagram
    Back end ->> Diff Storage: Give me the raw diff of this file
    Diff Storage ->> Back end: Raw diff

Backend

- First files rendered on page load

```
mermaid
sequenceDiagram
    participant Client
    participant Back end
    participant Authorization
    participant HAML
    participant Cache
    participant Database
    participant Diff storage
    participant Gitaly

    Client ->> Back end: Page load request
    Back end ->> Authorization: Check is good request
    alt Unauthorized
        Authorization ->> Back end: No!
        Back end ->> Client: 403 or 404
    else
        Authorization ->> Back end: Authorized.
        alt MR Diff
            Back end ->> Database: Get N files
            Database ->> Back end: Files
            Back end ->> Diff storage: Get diffs of N files
            Diff storage ->> Back end: Diffs
        else
            Back end ->> Gitaly: Get diffs of N files
            Gitaly ->> Back end: Diffs
        end
        loop Iterate through each diff file
            Back end ->> HAML: Render diff file
            HAML ->> Cache: Give me the cached rendered UI per file
            alt Cache miss
                Cache ->> HAML: Nada!
                HAML ->> Cache: Cache rendered UI per file
                Cache ->> HAML: Cached, rendered UI per file
            else
                Cache ->> HAML: Cached, rendered UI per file
            end
            HAML ->> Back end: Rendered UI
        end
        Back end ->> Client: Respond with application layout with rendered UI
    end


- Future files rendered and streamed to the front end

mermaid
sequenceDiagram
    participant Client
    participant Back end
    participant Authorization
```

```
participant HAML
participant Cache
participant Database
participant Diff storage
participant Gitaly

Client ->> Back end: Stream request
Back end ->> Authorization: Check is good request
alt All the possible unhappy paths
    Authorization ->> Back end: No!
    Back end ->> Client: 403
else
    Authorization ->> Back end: Authorized.
    alt MR Diff
        Back end ->> Database: Get files
        Database ->> Back end: Files
        Back end ->> Diff storage: Get diffs
        Diff storage ->> Back end: Diffs
    else
        Back end ->> Gitaly: Get diffs
        Gitaly ->> Back end: Diffs
    end
    loop Iterate through each diff file
        Back end ->> HAML: Render diff file
        HAML ->> Cache: Give me the cached rendered UI per file
        alt Cache miss
            Cache ->> HAML: Nada!
            HAML ->> Cache: Cache rendered UI per file
            Cache ->> HAML: Cached, rendered UI per file
        else
            Cache ->> HAML: Cached, rendered UI per file
        end
        HAML ->> Back end: Rendered UI
    end
    Back end ->> Client: Stream rendered UI per file
end
```

Web API

The Web API provides both internal and public access to the back end implementation for diffs.

Eventually, this diagram should expand (and possibly split) to show each endpoint that our application or a user could interface with, and what each of those endpoints expects and returns.

Note that this is separate from the Back End diagrams, which elaborate on business logic and implementation details.
The API endpoints are consumer-facing and so have different requirements and structures.

```
mermaid
sequenceDiagram
    actor Web User
    participant Endpoints
    participant Back end

    Web User ->> Endpoints: Give me the diff for [x] file
    Endpoints ->> Back end: User [u] is requesting [x] diff
    Back end ->> Endpoints: Here is the resolved, rendered UI for that diff
    Endpoints ->> Web User: "Do with this diff whatever you'd like to"
```

A complete, single render

```
mermaid
sequenceDiagram
actor User
participant UI
participant UX as Interaction handlers
participant FeApp as Front end behaviors
participant FeData as Data abstraction
participant FeNet as Network connectivity
participant API as Web API
participant BE as Back end
participant xxx
participant Cache
participant Database
participant Gitaly

User -->> BE: (MR page load)
BE ->> xxx: ???
xxx ->> Cache: ???
Cache ->> xxx: ???
xxx ->> Database: ???
Database ->> xxx: ???
xxx ->> Gitaly: ???
Gitaly ->> xxx: ???
xxx ->> BE: Rendered HTML
BE ->> User: A rendered diffs page for the MR
```

Accessibility

Reusable Rapid Diffs should be displayed in a way that is compliant with Web Content Accessibility Guidelines 2.1 level AA for web-based content and Authoring Tool Accessibility Guidelines 2.0 level AA for user interface.

We recognize that in order to have an accessible experience using diffs in the context of GitLab, we need to ensure the compliance both for displaying and interacting with diffs. That's

why the accessibility
audit and further recommendation will also consider Content Editor used feature for reviewing changes.

ATAG 2.0 AA

Giving the nature of diffs, the following guidelines will be our main focus:

1. Guideline A.2.1: (For the authoring tool user interface) Make alternative content available to authors
1. Guideline A.3.1: (For the authoring tool user interface) Provide keyboard access to authoring features
1. Guideline A.3.4: (For the authoring tool user interface) Enhance navigation and editing via content structure
1. Guideline A.3.6: (For the authoring tool user interface) Manage preference settings

HTML structure

The HTML structure of a diff should have support for assistive technology.
For this reason, a table could be a preferred solution as it allows to indicate logical relationship between the presented data and is easier to navigate for screen reader users with keyboard. Labeled columns will make sure that information such as line numbers can be associated with the edited piece of code.

Possible structure could include:

```
html
<table>
  <caption class="gl-sr-only">Changes for file index.js. 10 lines changed: 5 deleted, 5 added.</caption>
  <tr hidden>
    <th>Original line number: </th>
    <th>Diff line number: </th>
    <th>Line change:</th>
  </tr>
  <tr>
    <td>1234</td>
    <td></td>
    <td>.tree-time-ago ,</td>
  </tr>
  [·]
</table>
```

See WAI tutorial on tables for
more implementation guidelines.

Each file table should include a short summary of changes that will read out:

- total number of lines changed,

- number of added lines,
- number of removed lines.

The summary of the table content can be placed either within <caption> element, or before the table within an element referred as aria-describedby.
See <abbr>WAI</abbr> (Web Accessibility Initiative) for more information on both approaches:

- Nesting summary inside the <caption> element
- Using aria-describedby to provide a table summary

However, if such a structure will compromise other functional aspects of displaying a diff, more generic elements together with ARIA support can be used.

Visual indicators

It is important that each visual indicator should have a screen reader text
denoting the meaning of that indicator. When needed, use gl-sr-only (in conjunction with focus:gl-not-sr-only if needed)
class to make the element accessible by screen readers, but not by sighted users.

Some of the visual indicators that require alternatives for assistive technology are:

- + or red highlighting to be read as added
- - or green highlighting to be read as removed

High-level implementation

<!--
This section should contain enough information that the specifics of your
change are understandable. This may include API specs (though not always
required) or even code snippets. If there's any ambiguity about HOW your
proposal will be implemented, this is the place to discuss them.

If you are not sure how many implementation details you should include in the
blueprint, the rule of thumb here is to provide enough context for people to
understand the proposal. As you move forward with the implementation, you may
need to add more implementation details to the blueprint, as those may become
an important context for important technical decisions made along the way. A
blueprint is also a register of such technical decisions. If a technical
decision requires additional context before it can be made, you probably should
document this context in a blueprint. If it is a small technical decision that
can be made in a merge request by an author and a maintainer, you probably do
not need to document it here. The impact a technical decision will have is
another helpful information - if a technical decision is very impactful,
documenting it, along with associated implementation details, is advisable.

If it's helpful to include workflow diagrams or any other related images.
Diagrams authored in GitLab flavored markdown are preferred. In cases where
that is not feasible, images should be placed under images/ in the same
directory as the index.md for the proposal.

-->

Alternative Solutions

<!--
It might be a good idea to include a list of alternative solutions or paths considered,
although it is not required. Include pros and cons for
each alternative solution/path.

"Do nothing" and its pros and cons could be included in the list too.
-->

Historical context

Reusable Rapid Diffs introduce a paradigm shift in our approach to rendering diffs. Before this proposed architecture, we had two different approaches to rendering diffs:

1. Merge requests heavily utilized client-side rendering.
1. All other pages mainly used server-side rendering with additional behavior implemented in JavaScript.

In merge requests, most of the rendering work was done on the client:

- The backend would only generate a JSON response with diffs data.
- The client would be responsible for both drawing the diffs and reacting to user input.

This led to us adopting a
virtualized scrolling solution
for client-side rendering, which sped up drawing large diff file lists significantly.

Unfortunately, this came with downsides of a very high maintenance cost and
constant bugs.
The user experience also suffered because we couldn't show diffs right away
when you visited a page, and had to wait for the JSON response first.
Lastly, this approach went completely parallel to the server-rendered diffs used on other pages,
which resulted in two completely separate codebases for the diffs.

Summary of the alternative solutions attempted

Here is a list of the strategies we have adopted or simply tested in the past:

- Full Server Side Rendering (adopted and replaced by Vue app): before the Vue refactor of the Merge Request Changes tab, diffs were fully rendered on the server. This resulted in long waits before the page started to render.
- Frontend templates (Vue) Server Side Rendered (tested): results and impact weren't compelling and pointed in the direction of partial SSR. (PoC MR)
- Batch diffing (adopted): Break up the diffs into async paginated requests, increasing in size (slow start). Bootstrapping time unsatisfactory, perceived performance still involved a long time of a page without content.

- Virtual Scrolling (adopted): several known side-effects like inability to fully use native search functionality, interferences and weird behavior while scrolling to elements, overall strain on the browser to keep reflowing and painting. (Comparison with the proposed approach in this blueprint)
- Repository Commits details paginated if too large (adopted): As an interim solution, really large commit diffs in the repository are now paginated with negative impact in UX, hiding away files and changes through multiple pages.
- Micro Code Review Frontend PoC (tested): This approach was significantly different from the application design used in the past, so it was never seriously explored as a way forward. Parts of this design - like custom elements and a reliance on events - have been incorporated into alternative approaches. (Micro Code Review Frontend PoC)
- Streaming Diffs using a node server (tested): Combines streaming with a dedicated nodejs server. Percursor to the proposed SSR approach in this blueprint. (PoC: Streaming diffs app)

Proposed changes

These changes (indicated by an arbitrary name like "Design A") suggest a proposed final path forward for this blueprint, but have not yet been accepted as the authoritative content.

- Mark the highest hierarchical heading with your design name. If you are changing multiple headings at the same level, make sure to mark them all with the same name. This will create a high-level table of contents that is easier to reason about.

Front end (Design A)

High-level implementation

NOTE:
This draft proposal suggests one potential front end architecture which may not be chosen. It is not necessarily mutually exclusive with other proposed designs.

(See New Diffs: Technical Architecture Design for nicer visuals of this chart)

```
mermaid
flowchart TB
    classDef sticky fill:d0cabf, color:black
    stickyMetricsA>"Metrics 3, 4, & 5 apply to<br>the entire front end application"]

    stickyMetricsA -.- fe
    fe

    Socket((WebSocket))

    be

subgraph fe [Front End]
    stickyMetricsB>"Metrics 1 & 2 apply<br>to all UI elements"]
    stickyInbound>"All data is formatted precisely<br>how the UI needs to interact with it"]
    stickyOutbound>"All data is formatted precisely<br>how the back end expects it"]
    stickyIdb>"Long-term.
```

```
    e.g. diffs, MRs, emoji, notes, drafts, user-only data<br>like file reviews, collapse
states, etc."]
    stickySession>"Session-term.

    e.g. selected tab, scroll position,<br>temporary changes to user settings, etc."]

    Events([Event Hub])
    UI[UI]
    uiState((Local State))
    Logic[Application Logic]
    Normalizer[Data Normalizer]
    Inbound{{Inbound Contract}}
    Outbound{{Outbound Contract}}
    Data[Data Access]
    idb((indexedDB))
    session((sessionStorage))
    Network[Network Access]
end

subgraph be [Back End]
    stickyApi>"A large list of defined actions a<br>Diffs/Merge Request UI could perform.

    e.g.: <code>mergeRequest:notes:saveDraft</code>
or<br><code>mergeRequest:changeStatus</code> (with <br><code>status: 'draft'</code> or
<code>status: 'ready'</code>, etc.).

    Must not expose any implementation detail,<br>like models, storage structure, etc."]
    API[Activities API]
    unk[\"?"/]

    API -.- stickyApi
end

    %% Make stickies look like paper sort of?
    class
stickyMetricsA,stickyMetricsB,stickyInbound,stickyOutbound,stickyIdb,stickySession,stickyApi
sticky

    UI <--> uiState
    stickyMetricsB -.- UI
    Network ~~~ stickyMetricsB

    Logic <--> Normalizer

    Normalizer --> Outbound
    Outbound --> Data
    Inbound --> Normalizer
    Data --> Inbound

    Inbound -.- stickyInbound
```

```
    Outbound -.- stickyOutbound

    Data <--> idb
    Data <--> session
    idb -.- stickyIdb
    session -.- stickySession

    Events <--> UI
    Events <--> Logic
    Events <--> Data
    Events <--> Network

    Network --> Socket --> API --> unk
```

SECTION: engineering/architecture/design-documents/rapid_diffs/features.md

This is an appendix to the Reusable Rapid Diffs document.

Below is a complete list of features for merge request and commit diffs grouped by diff viewers (Code, Image, Other).

· · available in both MR and Commit views.

Features	Code	Image	Other
Filename	·	·	·
Copy file path	·	·	·
Collapse and expand file	·	·	·
File stats	·	·	·
Lines changed (0 for blobs)	·	·	·
Permissions changed	·	·	·
CRUD comment on file	·	·	·
View file link	·	·	·
Mark as viewed	MR	MR	MR
Hide all comments	MR	MR	MR
Show full file (expand all lines)	MR		
Open in Web IDE link	MR		
Line link	·		
Edit file link	·		
Code highlight (multiple themes)	·		
Expand lines	·		
CRUD comment on specific line	Commit		
CRUD comment on line range	MR		
Draft comment on line range	MR		
Code quality highlights	·		
Test coverage highlights	·		
Hide whitespace changes	·		
Auto-collapse large file	·		

| View as raw | Commit | | |
| Side by side view | | · | |

SECTION: engineering/architecture/design-documents/rate_limiting/_index.md

{{< engineering/design-document-header >}}

Summary

Introducing reasonable application limits is a very important step in any SaaS platform scaling strategy. The more users a SaaS platform has, the more important it is to introduce sensible rate limiting and policies enforcement that will help to achieve availability goals, reduce the problem of noisy neighbours for users and ensure that they can keep using a platform successfully.

This is especially true for GitLab.com. Our goal is to have a reasonable and transparent strategy for enforcing application limits, which will become a definition of a responsible usage, to help us with keeping our availability and user satisfaction at a desired level.

We've been introducing various application limits for many years already, but we've never had a consistent strategy for doing it. What we want to build now is a consistent framework used by engineers and product managers, across entire application stack, to define, expose and enforce limits and policies.

Lack of consistency in defining limits, not being able to expose them to our users, support engineers and satellite services, has negative impact on our productivity, makes it difficult to introduce new limits and eventually prevents us from enforcing responsible usage on all layers of our application stack.

This blueprint has been written to consolidate our limits and to describe the vision of our next rate limiting and policies enforcement architecture.

Goals

Implement a next architecture for rate limiting and policies definition.

Challenges

- We have many ways to define application limits, in many different places.
- It is difficult to understand what limits have been applied to a request.
- It is difficult to introduce new limits, even more to define policies.
- Finding what limits are defined requires performing a codebase audit.
- We don't have a good way to expose limits to satellite services like Registry.
- We enforce a number of different policies via opaque external systems (Pipeline Validation Service, Bouncer, Watchtower, Cloudflare, HAProxy).

- There is not standardized way to define policies in a way consistent with defining limits.
- It is difficult to understand when a user is approaching a limit threshold.
- There is no way to automatically notify a user when they are approaching thresholds.
- There is no single way to change limits for a namespace / project / user / customer.
- There is no single way to monitor limits through real-time metrics.
- There is no framework for hierarchical limit configuration (instance / namespace / subgroup / project).
- We allow disabling rate-limiting for some marquee SaaS customers, but this increases a risk for those same customers. We should instead be able to set higher limits.

Opportunity

We want to build a new framework, making it easier to define limits, quotas and policies, and to enforce / adjust them in a controlled way, through robust monitoring capabilities.

<!-- markdownlint-disable MD029 -->

1. Build a framework to define and enforce limits in GitLab Rails.
2. Build an API to consume limits in satellite service and expose them to users.
3. Extract parts of this framework into a dedicated GitLab Limits Service.

<!-- markdownlint-enable MD029 -->

The most important opportunity here is consolidation happening on multiple levels:

1. Consolidate on the application limits tooling used in GitLab Rails.
1. Consolidate on the process of adding and managing application limits.
1. Consolidate on the behavior of hierarchical cascade of limits and overrides.
1. Consolidate on the application limits tooling used across entire application stack.
1. Consolidate on the policies enforcement tooling used across entire company.

Once we do that we will unlock another opportunity: to ship the new framework / tooling as a GitLab feature to unlock these consolidation benefits for our users, customers and entire wider community audience.

Limits, quotas and policies

This document aims to describe our technical vision for building the next rate limiting architecture for GitLab.com. We refer to this architectural evolution as "the next rate limiting architecture", but this is a mental shortcut, because we actually want to build a better framework that will make it easier for us to manage not only rate limits, but also quotas and policies.

Below you can find a short definition of what we understand by a limit, by a quota and by a policy.

- Limit: A constraint on application usage, typically used to mitigate

  risks to performance, stability, and security.
  - Example: API calls per second for a given IP address
  - Example: git clone events per minute for a given user
  - Example: maximum artifact upload size of 1 GB
- Quota: A global constraint in application usage that is aggregated across an entire namespace over the duration of their billing cycle.
  - Example: 400 compute minutes per namespace per month
  - Example: 10 GB transfer per namespace per month
- Policy: A representation of business logic that is decoupled from application code. Decoupled policy definitions allow logic to be shared across multiple services and/or "hot-loaded" at runtime without releasing a new version of the application.
  - Example: decode and verify a JWT, determine whether the user has access to the given resource based on the JWT scopes and claims
  - Example: deny access based on group-level constraints (such as IP allowlist, SSO, and 2FA) across all services

Technically, all of these are limits, because rate limiting is still "limiting", quota is usually a business limit, and policy limits what you can do with the application to enforce specific rules. By referring to a "limit" in this document we mean a limit that is defined to protect business, availability and security.

Framework to define and enforce limits

First we want to build a new framework that will allow us to define and enforce application limits, in the GitLab Rails project context, in a more consistent and established way. In order to do that, we will need to build a new abstraction that will tell engineers how to define a limit in a structured way (presumably using YAML or Cue format) and then how to consume the limit in the application itself.

We already do have many limits defined in the application, we can use them to triangulate to find a reasonable abstraction that will consolidate how we define, use and enforce limits.

We envision building a simple Ruby library here (we can add it to LabKit) that will make it trivial for engineers to check if a certain limit has been exceeded or not.

yaml
name: mylimitname
actors: user
context: project, group, pipeline
type: rate / second
group: pipeline::execution
limits:
  warn: 2B / day
  soft: 100k / s
  hard: 500k / s

```
ruby
Gitlab::Limits::RateThreshold.enforce(:mylimitname) do |threshold|
  actor   = currentuser
  context = currentproject

  threshold.available do |limit|
    ...
  end

  threshold.approaching do |limit|
    ...
  end

  threshold.exceeded do |limit|
    ...
  end
end
```

In the example above, when mylimitname is defined in YAML, engineers will be check the current state and execute appropriate code block depending on the past usage / resource consumption.

Things we want to build and support by default:

1. Comprehensive dashboards showing how often limits are being hit.
1. Notifications about the risk of hitting limits.
1. Automation checking if limits definitions are being enforced properly.
1. Different types of limits - time bound / number per resource etc.
1. A panel that makes it easy to override limits per plan / namespace.
1. Logging that will expose limits applied in Kibana.
1. An automatically generated documentation page describing all the limits.

Support rate limits based on resources used

One of the problems of our rate limiting system is that values are static (e.g. 100 requests per minutes) and irrespective of the complexity or resources used by the operation. For example:

- Firing 100 requests per minute to fetch a simple resource can have very different implications than creating a CI pipeline.
- Each pipeline creation action can perform very differently depending on the pipeline being created (small MR pipeline VS large scheduled pipeline).
- Paginating resources after an offset of 1000 starts to become expensive on the database.

We should allow some rate limits to be defiened as computing score / period where for computing score we calculate the milliseconds accumulated (for all requests executed and inflight) within a given period (for example: 1 minute).

This way if a user is sending expensive requests they are likely to hit the rate limit earlier.

API to expose limits and policies

Once we have an established a consistent way to define application limits we can build a few API endpoints that will allow us to expose them to our users, customers and other satellite services that may want to consume them.

Users will be able to ask the API about the limits / thresholds that have been set for them, how often they are hitting them, and what impact those might have on their business. This kind of transparency can help them with communicating their needs to customer success team at GitLab, and we will be able to communicate how the responsible usage is defined at a given moment.

Because of how GitLab architecture has been built, GitLab Rails application, in most cases, behaves as a central enterprise service bus (ESB) and there are a few satellite services communicating with it. Services like container registry, GitLab Runners, Gitaly, Workhorse, KAS could use the API to receive a set of application limits those are supposed to enforce. This will still allow us to define all of them in a single place.

We should, however, avoid the possible negative-feedback-loop, that will put additional strain on the Rails application when there is a sudden increase in usage happening. This might be a big customer starting a new automation that traverses our API or a Denial of Service attack. In such cases, the additional traffic will reach GitLab Rails and subsequently also other satellite services. Then the satellite services may need to consult Rails again to obtain new instructions / policies around rate limiting the increased traffic. This can put additional strain on Rails application and eventually degrade performance even more. In order to avoid this problem, we should extract the API endpoints to separate service (see the section below) if the request rate to those endpoints depends on the volume of incoming traffic. Alternatively we can keep those endpoints in Rails if the increased traffic will not translate into increase of requests rate or increase in resources consumption on these API endpoints on the Rails side.

Decoupled Limits Service

At some point we may decide that it is time to extract a stateful backend responsible for storing metadata around limits, all the counters and state required, and exposing API, out of Rails.

It is impossible to make a decision about extracting such a decoupled limits service yet, because we will need to ship more proof-of-concept work, and concrete iterations to inform us better about when and how we should do that. We will depend on the Evolution Architecture practice to guide us towards either extracting Decoupled Limits Service or not doing that at all.

As we evolve this blueprint, we will document our findings and insights about how this service should look like, in this section of the document.

GitLab Policy Service

Disclaimer: Extracting a GitLab Policy Service might be out of scope
of the current workstream organized around implementing this blueprint.

Not all limits can be easily described in YAML. There are some more complex
policies that require a bit more sophisticated approach and a declarative
programming language used to enforce them. One example of such a language might be
Rego language.
It is a standardized way to define policies in
OPA - Open Policy Agent. At GitLab we are
already using OPA in some departments. We envision the need to additional
consolidation to not only consolidate on the tooling we are using internally at
GitLab, but to also transform the Next Rate Limiting Architecture into
something we can make a part of the product itself.

Today, we already do have a policy service we are using to decide whether a
pipeline can be created or not. There are many policies defined in
Pipeline Validation Service.
There is a significant opportunity here in